

闭包与作用域中英对照笔记

A closure is a function bundled together with the environment it captured at birth. In JavaScript, every function keeps a live reference to the scope where it was defined, so it can still read those variables long after the outer call has returned. This is the mechanism behind counters, memoization caches, and module patterns.

闭包是函数与其出生时词法环境的组合体。每个函数都持有定义它时的作用域引用。因此外层调用返回之后，内层函数仍能读写那些变量。计数器、缓存与模块模式都建立在这个机制上。

词法作用域

Scope in JavaScript is lexical: it is decided by where code is written, not by where the function is called from. When the engine looks up a name, it walks the chain of enclosing scopes outward until it finds a binding. That chain is fixed at parse time, which is why closures capture variables, not values.

作用域是词法的，由代码书写的位置决定，而不是由调用位置决定。引擎查找名字时会沿着定义位置向外层作用域逐级寻找，直到命中绑定。这条链在解析期就固定了，所以闭包捕获的是变量本身，而不是当时的值快照。

一个经典陷阱是在循环里用 `var` 声明循环变量。回调函数捕获的是同一个绑定，因此所有回调读到的都是循环结束后的值。换成 `let` 或 `const`，每次迭代都会建立新的绑定，问题随之消失。理解这一点比背下答案更重要。

2. Practical closure patterns

`Memoize` wraps a function with a cache keyed by arguments; the cache lives in the wrapper's closure and stays private. Once hooks resolve an initializer exactly one time. Both patterns rely on the same property: hidden mutable state with a controlled interface.

记忆化是一个以参数为键的缓存。包裹函数，缓存藏在闭包里，外界碰不到。一次性开关，保证初始化逻辑只执行一次。这两个惯用法共享同一性质：把可变状态藏进闭包，只暴露受控接口。

Closures are not free. A captured scope keeps every binding in it alive as long as the inner function is reachable, which is a common source of accidental memory retention. Capture narrowly: take what you need, let the rest go.

闭包的代价与边界

闭包的代价是生命周期只要内层函数还可达被捕获的作用域连同其中的所有绑定就都不能回收大型对象被无意间捕获是最常见的内存滞留原因审查代码时值得专门检查一次回调究竟捕获了什么？

边界守则捕获要窄把真正需要的外部值收进局部常量再让闭包捕获这个常量；不需要的引用不要顺手带进闭包守则简单却能把大多数滞留问题挡在设计期。

最后用一句话收束闭包让函数拥有记忆而工程师的工作是决定它记住什么忘记什么中英对照的这份笔记到此结束两套表述描述的是同一个机制。